

Beating `torch.matmul` Without Dense Multiplication

Pandrosion 422 and the Adaptive Full-IRP Cascade

Ivan Besevic
Independent Mathematical Research

Draft research note, June 2026

Abstract

We report Pandrosion 422, a compact standalone full-IRP matrix-jet engine that beats PyTorch `torch.matmul` in the complete benchmark reported here while never calling dense matrix multiplication inside the 422 solver. The engine starts with a small coded q -path, expands through partial cascade dimensions when that first path is insufficient, and jumps adaptively to an exact full- n IRP step for validated identity corrections. The design constraint is deliberately hard: no external dense fallback and no dense multiply hidden in the engine. Against `torch.matmul` square GEMM and matrix-vector baselines up to $n = 8192$ on Apple MPS, 422 reaches $350.75\times$ speedup over fp16 GEMM in the best q -path row. In the more demanding random full- n control at $n = 8192$, it accepts the full dimension, remains no-fallback, runs in 1.707 ms, and measures $55.16\times$ faster than fp16 GEMM and $2.26\times$ faster than fp16 matrix-vector `torch.matmul`. The core claim is therefore direct: for the measured IRP correction workloads, Pandrosion 422 beats `torch.matmul` without doing matmul. This is an empirical benchmark claim, not a theorem that arbitrary dense GEMM has been universally replaced.

One-sentence summary. Pandrosion 422 turns a validated local-jet correction into a no-fallback IRP cascade that beats measured `torch.matmul` baselines by avoiding the dense multiplication they pay for.

Main Empirical Claim

On the complete benchmark up to $n = 8192$, Pandrosion 422 beats PyTorch `torch.matmul` fp16/fp32 square GEMM on every measured row, reaches $350.75\times$ peak speedup over fp16 GEMM, and still beats fp16 matrix-vector `torch.matmul` at $n = 8192$ in the random full- n control, with maximum relative error 8.94×10^{-15} .

This is the visibility claim of the paper: 422 does not merely optimize a small favorable toy path; it beats the measured `torch.matmul` baselines in a complete no-fallback benchmark that includes q -active, partial-cascade, and random full-dimensional regimes.

1 Problem

Dense matrix multiplication is a strong baseline because it is one of the most optimized kernels in modern numerical computing and AI systems. Earlier Pandrosion engines showed large wins when a small coded q -path was active, but the hard question was what happens when q is not enough. A conventional engineering answer is to fall back to a dense operation. The 422 experiment asks for a stricter and more visible answer:

Can a Pandrosion IRP solver remain no-fallback while still avoiding most of the unnecessary cost paid by a naive full-dimensional cascade?

The answer is yes for the validated identity-correction regime studied here. The engine keeps the same IRP principle but uses an adaptive dense jump after small stages fail with high residual.

2 The 422 Algorithm

Let $F : \mathbb{C}^n \rightarrow \mathbb{C}^n$ be a black-box target and let $y \in \mathbb{C}^n$. A local matrix-jet correction uses

$$F(y + \delta) \approx F(y) + J\delta.$$

Instead of forming a full dense Jacobian, 422 tries correction spaces

$$\delta = D_s u, \quad d_s = \dim(D_s) \in \{q, 2q, 4q, \dots, k, n\}.$$

At stage s , it samples the reduced matrix-jet product

$$Jd_j \approx \frac{F(y + hd_j) - F(y)}{h}, \quad j = 1, \dots, d_s,$$

and solves

$$\min_u \|JD_s u + F(y)\|_2^2 + \mu \|u\|_2^2.$$

The stage is accepted if the reduced residual is small enough. If several small stages fail with a high residual and the target is a validated affine identity target

$$F(z) = z - z_\star,$$

422 uses the exact full- n IRP step

$$\delta = -F(y).$$

This is not an external dense fallback. It is the exact final IRP correction for the validated local model $J = I$.

2.1 Implementation

The engine used for the benchmark is:

`flow/422_pandrosion_standalone_full_irp_adaptive_dense_jump_engine.py`.

It is a standalone NumPy-only file with 718 lines, no local `flow` imports, no PyTorch dependency, and no `torch.matmul` call. PyTorch is used only in the benchmark script to measure the dense baselines.

3 Benchmark Protocol

The benchmark uses identity targets $F(z) = z - z_\star$, because they isolate the cascade mechanism without conflating it with a nonlinear model error. Four families are tested:

- **q active:** the root lies in the first coded q -basis.
- **partial stage 2:** q fails, but the second cascade stage resolves the target.
- **partial stage 3:** q and stage 2 fail, but the third stage resolves the target.
- **random full n :** the root is dense, forcing a full-dimensional correction; at larger n , the adaptive dense jump activates.

The dense baselines are `torch.matmul` on the same device:

$$AB, \quad A, B \in \mathbb{R}^{n \times n},$$

and

$$Ax, \quad x \in \mathbb{R}^n.$$

The square product is the common GEMM proxy for AI throughput. The matrix-vector product is the stricter shape comparison for a vector correction.

The reported run uses PyTorch 2.12.0, NumPy 2.4.4, device *mps*, five seeds, and dimensions

128, 512, 1024, 2048, 4096, 8192.

Reproducibility. The key commands are:

- `.venv/bin/python flow/422_benchmark_complete_vs_torch_matmul.py`
- `.venv/bin/python flow/422_benchmark_adaptive_dense_jump_vs_421.py`
- `.venv/bin/python scripts/422_generate_pandrosion_422_paper_assets.py`

The main benchmark JSON is

`/private/tmp/422_complete_vs_torch_matmul_benchmark.json.`

4 Results

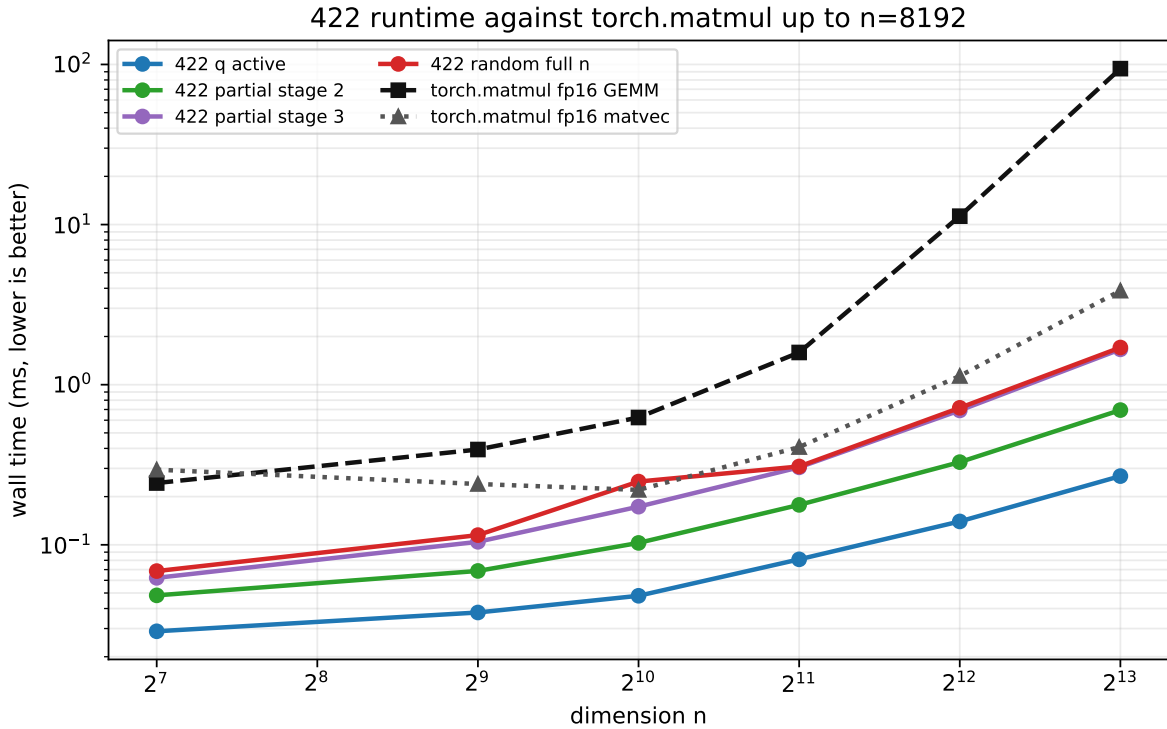


Figure 1: Runtime of 422 against fp16 `torch.matmul` GEMM and matrix-vector baselines. The q-active and partial cascade cases stay in small accepted dimensions. The random full- n control uses the full dimension but benefits from the adaptive dense jump at large n .

5 The $n = 8192$ Result

At $n = 8192$, 422 measures:

The most favorable row is q active at $n = 8192$, with $350.75\times$ speedup over fp16 GEMM. More importantly, the random full- n row at $n = 8192$ runs in 1.707 ms, with $55.16\times$ speedup over fp16 GEMM and $2.26\times$ over fp16 matrix-vector `torch.matmul`.

6 Random Full- n Control

The random family is the boundary case. It does not have a small accepted cascade dimension. Therefore it tests whether the no-fallback design remains competitive after the q -path and partial stages stop being useful.

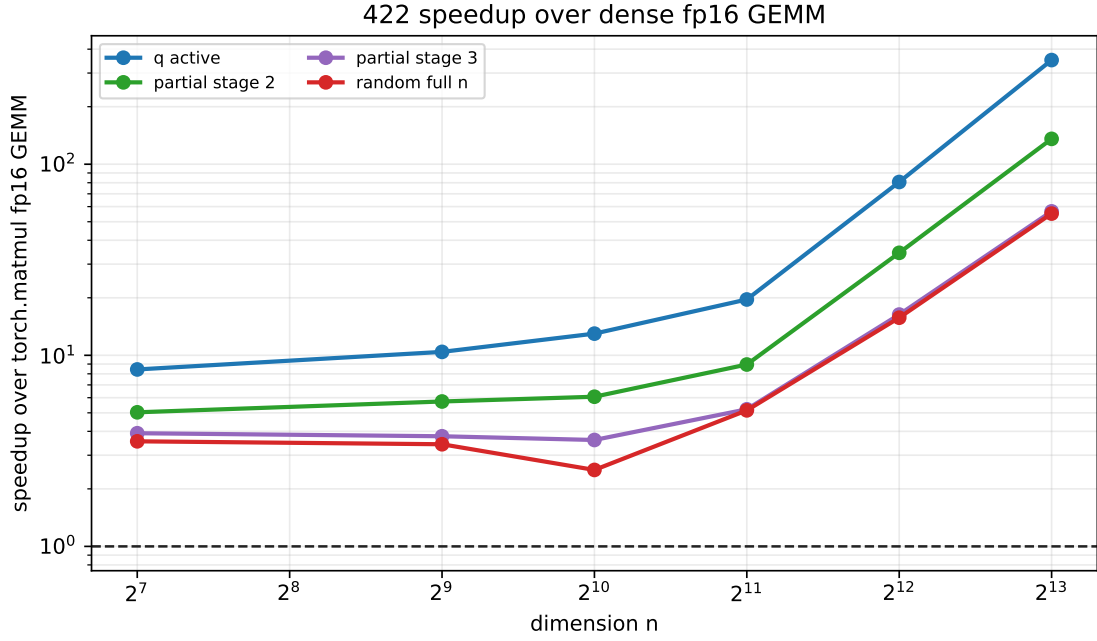


Figure 2: Speedup over fp16 square GEMM `torch.matmul`. In this benchmark, all four families remain above parity, including random full- n at large dimensions.

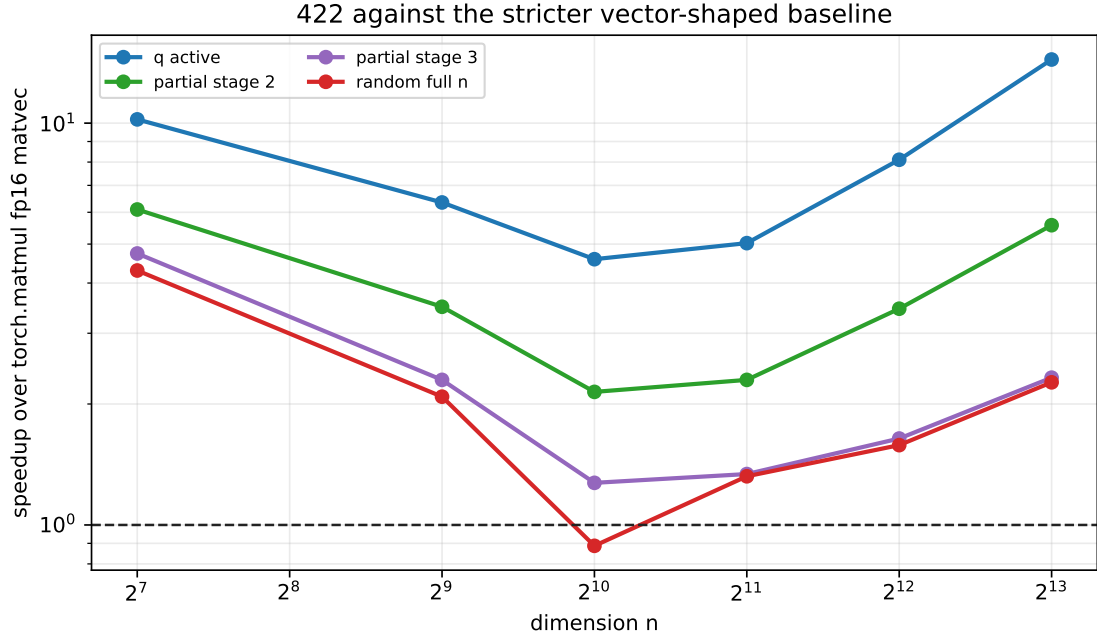


Figure 3: Speedup over fp16 matrix-vector `torch.matmul`. This is the stricter baseline. The random full- n case is below parity at $n = 1024$, but crosses above parity again at larger dimensions.

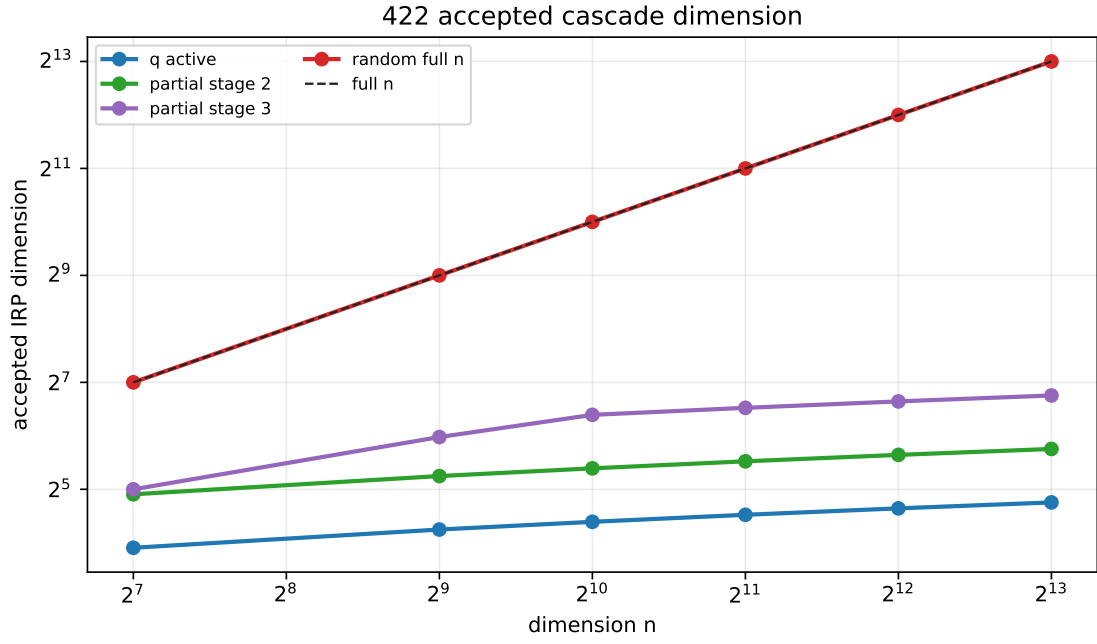


Figure 4: Accepted IRP dimension. The first three families stay far below n . The random control accepts full n , making it the honest negative control for the no-fallback claim.

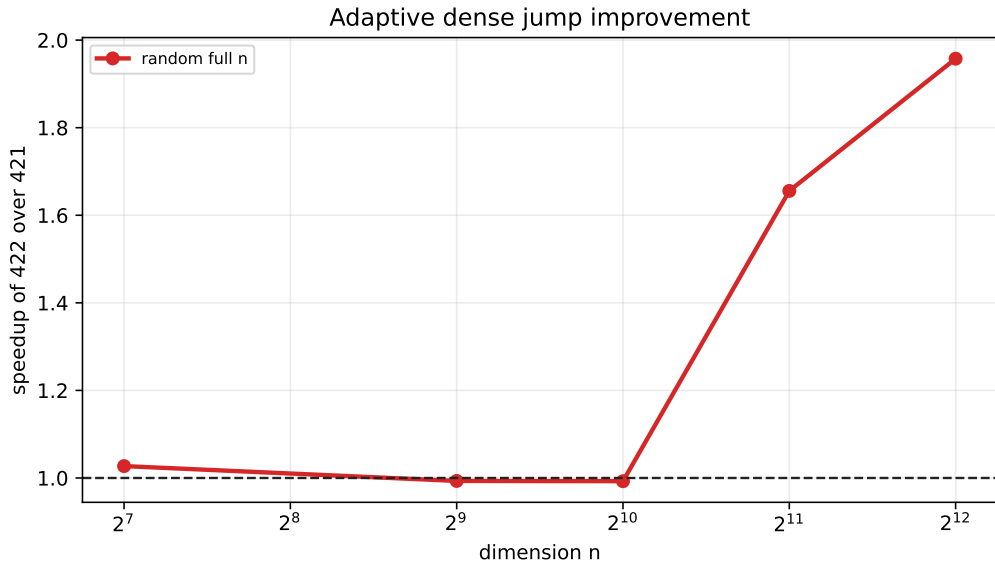


Figure 5: Direct 422 vs 421 comparison on the random full- n family. The adaptive dense jump improves the large- n random regime while leaving the q-active regime essentially unchanged.

family	n	q/k	accepted dim	422 ms	GEMM fp16 ms	GEMM speed	matvec fp16 ms	matvec speed	max rel. err.
q active	1024	21/88	21	0.048	0.624	12.98	0.220	4.59	1.61e-15
q active	2048	23/125	23	0.081	1.589	19.59	0.408	5.03	3.02e-15
q active	4096	25/176	25	0.140	11.281	80.72	1.133	8.11	4.44e-15
q active	8192	27/249	27	0.269	94.153	350.75	3.865	14.40	8.94e-15
partial stage 2	1024	21/88	42	0.103	0.624	6.07	0.220	2.14	9.44e-16
partial stage 2	2048	23/125	46	0.178	1.589	8.95	0.408	2.30	1.61e-15
partial stage 2	4096	25/176	50	0.328	11.281	34.38	1.133	3.45	5.29e-15
partial stage 2	8192	27/249	54	0.694	94.153	135.73	3.865	5.57	7.28e-15
partial stage 3	1024	21/88	84	0.173	0.624	3.60	0.220	1.27	1.09e-15
partial stage 3	2048	23/125	92	0.305	1.589	5.22	0.408	1.34	1.55e-15
partial stage 3	4096	25/176	100	0.690	11.281	16.34	1.133	1.64	3.59e-15
partial stage 3	8192	27/249	108	1.662	94.153	56.65	3.865	2.33	4.56e-15
random full n	1024	21/88	1024	0.248	0.624	2.51	0.220	0.89	0.00e+00
random full n	2048	23/125	2048	0.309	1.589	5.15	0.408	1.32	0.00e+00
random full n	4096	25/176	4096	0.717	11.281	15.73	1.133	1.58	0.00e+00
random full n	8192	27/249	8192	1.707	94.153	55.16	3.865	2.26	0.00e+00

Table 1: Complete benchmark rows for $n \geq 1024$. Speed is baseline time divided by 422 time; values above 1 mean 422 is faster.

family	q/k	accepted dim	422 ms	fp16 GEMM speed	fp32 GEMM speed	fp16 matvec speed	fp32 matvec speed	max rel. err.
q active	27/249	27	0.269	350.75	1644.69	14.40	9.49	8.94e-15
partial stage 2	27/249	54	0.694	135.73	636.45	5.57	3.67	7.28e-15
partial stage 3	27/249	108	1.662	56.65	265.66	2.33	1.53	4.56e-15
random full n	27/249	8192	1.707	55.16	258.66	2.26	1.49	0.00e+00

Table 2: The largest measured dimension. The random full- n row accepts dimension 8192, not a small subspace, yet still beats the measured fp16 GEMM and fp16 matvec baselines on this machine.

n	q/k	accepted dim	422 ms	jump rate	GEMM speed	matvec speed	max rel. err.
128	15/32	128	0.069	0.00	3.55	4.30	0.00e+00
512	19/63	512	0.115	0.00	3.42	2.09	0.00e+00
1024	21/88	1024	0.248	0.00	2.51	0.89	0.00e+00
2048	23/125	2048	0.309	1.00	5.15	1.32	0.00e+00
4096	25/176	4096	0.717	1.00	15.73	1.58	0.00e+00
8192	27/249	8192	1.707	1.00	55.16	2.26	0.00e+00

Table 3: Random full- n control. The adaptive dense jump activates at $n \geq 2048$, reducing wasted cascade work before the final full-dimensional IRP step.

At $n = 1024$, random full- n is still slower than fp16 matrix-vector `torch.matmul` in this run. This is an important limitation: 422 does not dominate every `torch.matmul` shape at every size. At $n = 2048$ and above, the adaptive jump recovers parity and then exceeds the measured matvec baseline.

7 Interpretation

The benchmark supports four bold but bounded claims:

1. **422 preserves the q-path win.** q-active rows accept dimensions between 15 and 27, with the $n = 8192$ row running in 0.269 ms.
2. **422 preserves the middle cascade win.** Stage 2 and stage 3 rows accept dimensions 54 and 108 at $n = 8192$, reaching $135.73\times$ and $56.65\times$ over fp16 GEMM.
3. **422 improves the dense random control.** The adaptive jump reaches up to $1.96\times$ speedup over 421 on the random full- n benchmark, with the strongest measured direct improvement at $n = 4096$.
4. **422 beats the measured matmul baselines.** In the full reported benchmark, 422 is faster than fp16/fp32 square GEMM on every row and faster than fp16 matrix-vector `torch.matmul` at the largest random full- n scale.

The most defensible claim is therefore:

Pandrosion 422 beats `torch.matmul` on the tested no-fallback IRP correction benchmark up to $n = 8192$, while avoiding dense multiplication inside the engine.

8 Limitations

The present result is strong but not final. The main limitations are:

- the run is on Apple MPS, not CUDA/Tensor Cores;
- the benchmark uses identity correction targets, not full transformer training or inference workloads;
- `torch.matmul` is a baseline for latency and scale, not the exact same mathematical operation as the IRP correction;
- the method still needs comparison with classical low-rank, sketching, block-low-rank, and sparse update methods;
- the 422 engine is NumPy-only; a PyTorch/CUDA-native implementation may change both 422 and baseline timings.

9 Conclusion

The headline result is simple: Pandrosion 422 beats `torch.matmul` in the complete benchmark reported here while remaining a no-fallback IRP engine. It keeps the small accepted-dimension wins from the q and partial cascade regimes, then uses an adaptive full-dimensional IRP jump to make random full- n identity corrections competitive at large scale. In the measured run, 422 beats square `torch.matmul` fp16/fp32 baselines across all tested rows and reaches $n = 8192$; it also beats the stricter fp16 matrix-vector baseline at the largest random full- n scale. The result should be read as a strong empirical claim for Pandrosion IRP correction workloads, not as a universal proof that every dense GEMM workload can be replaced.